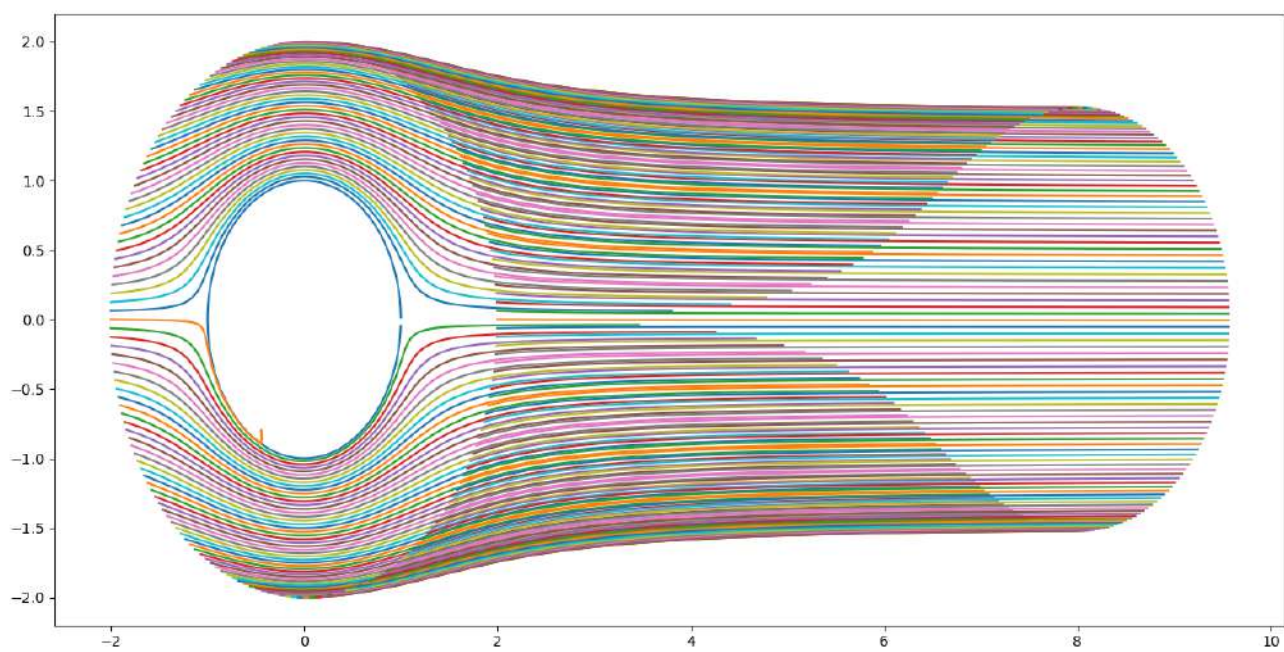
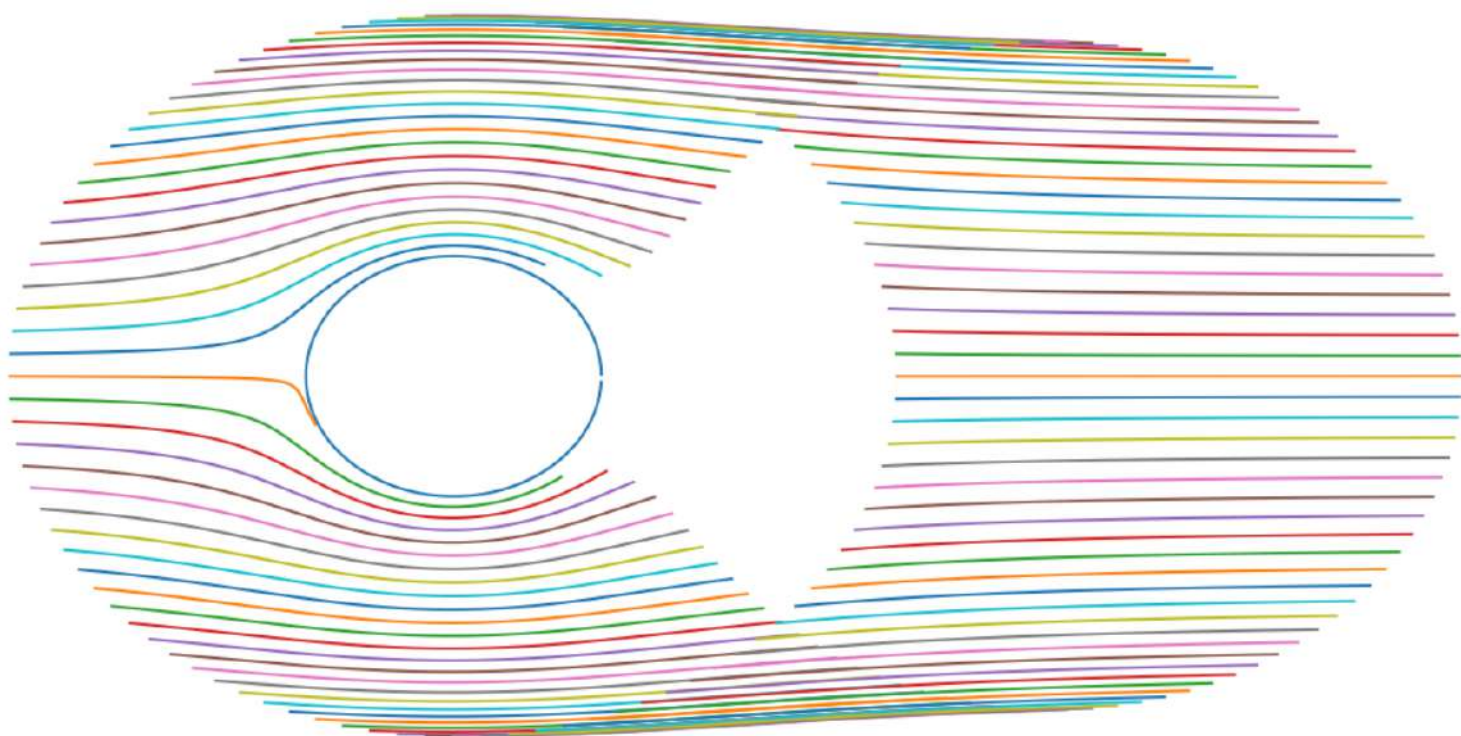
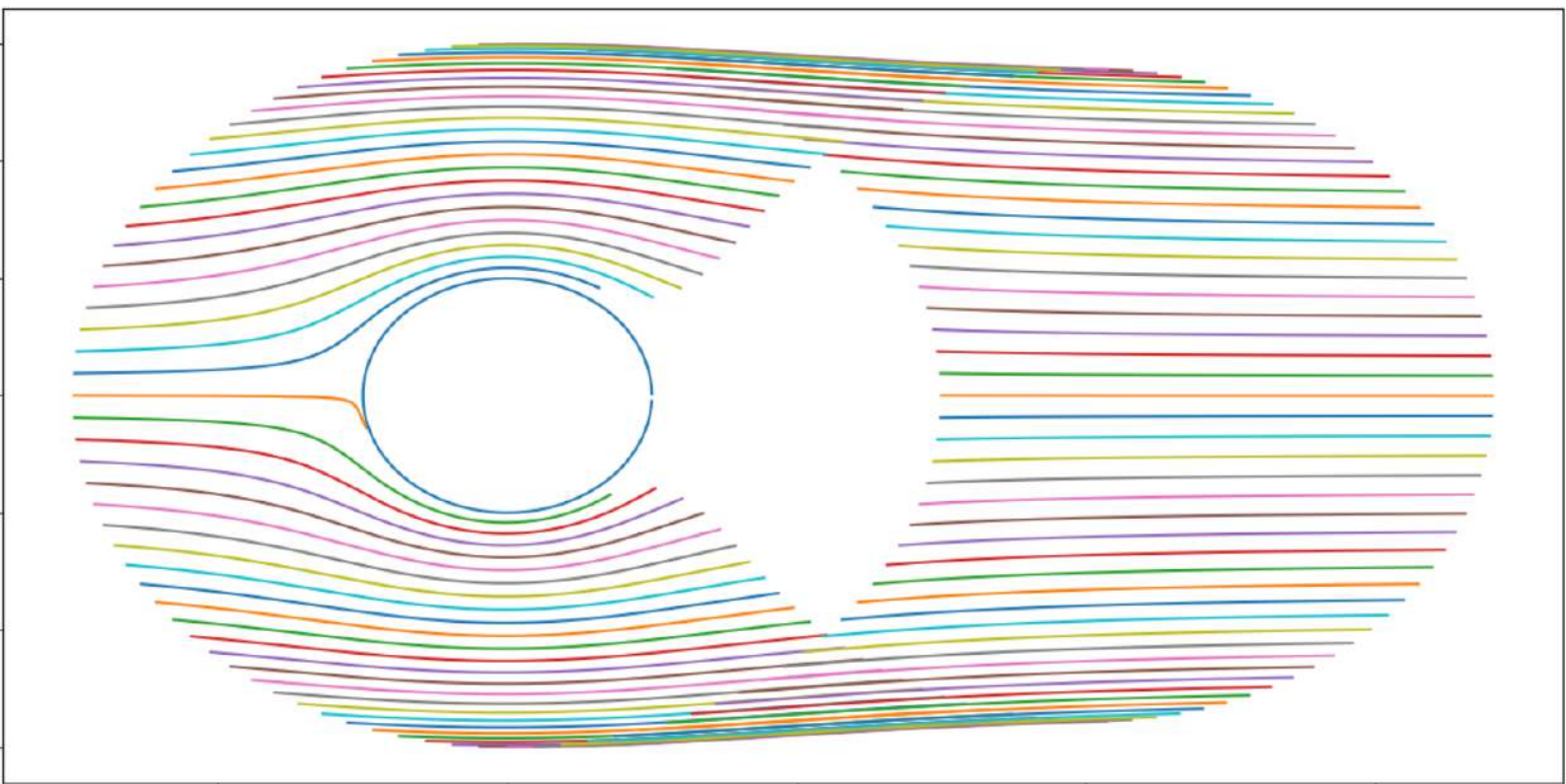
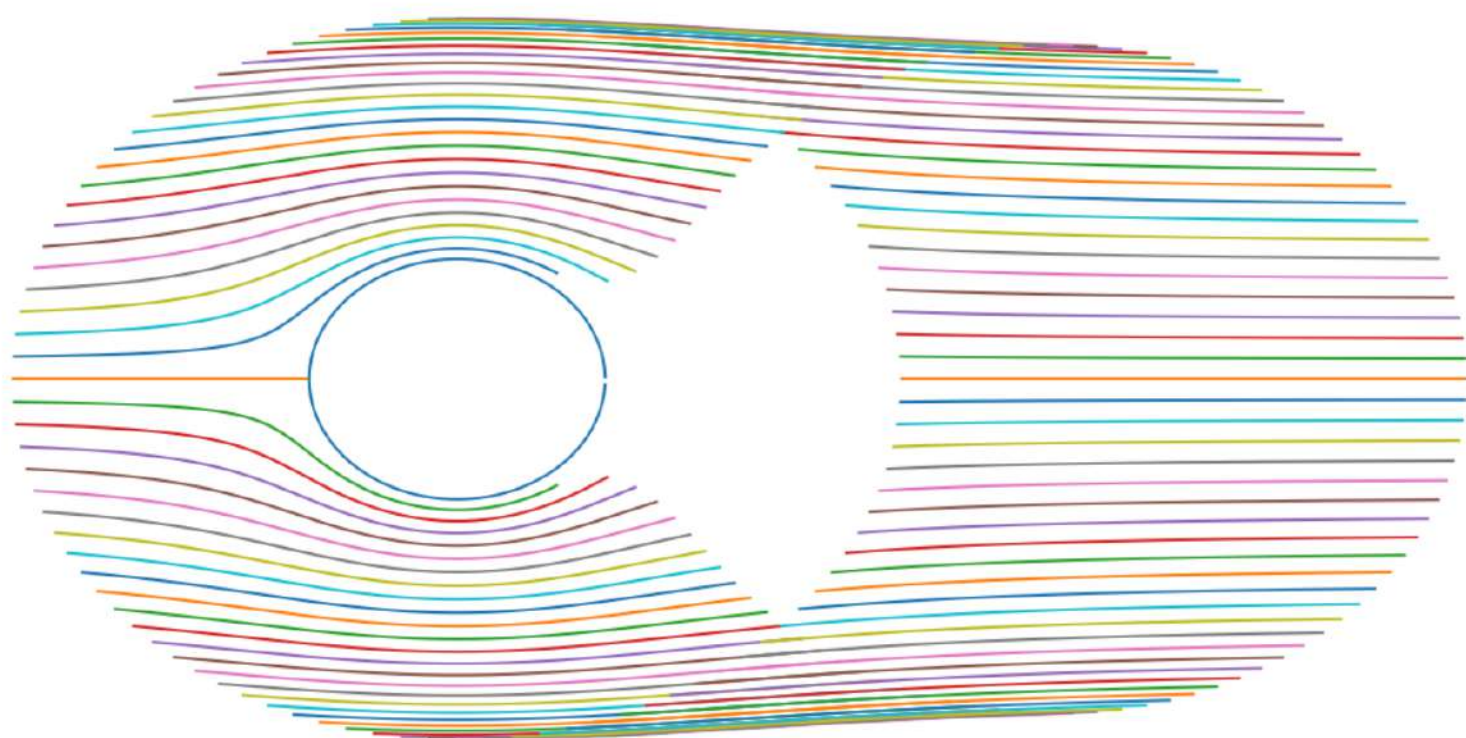
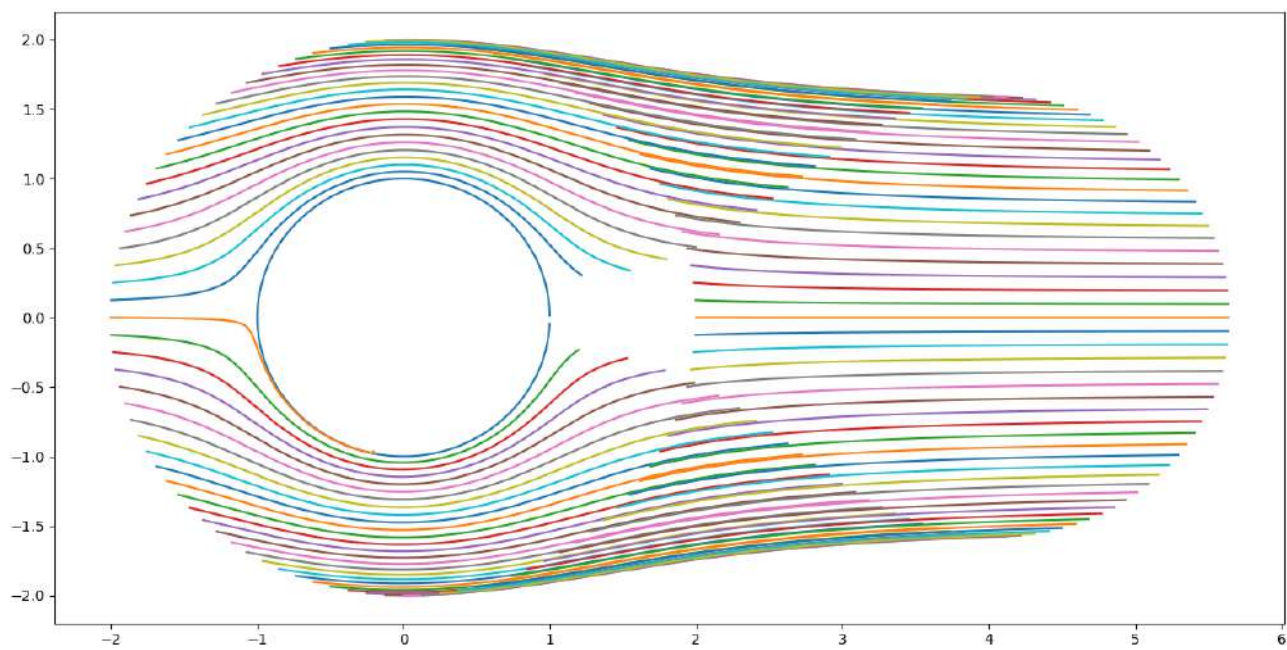


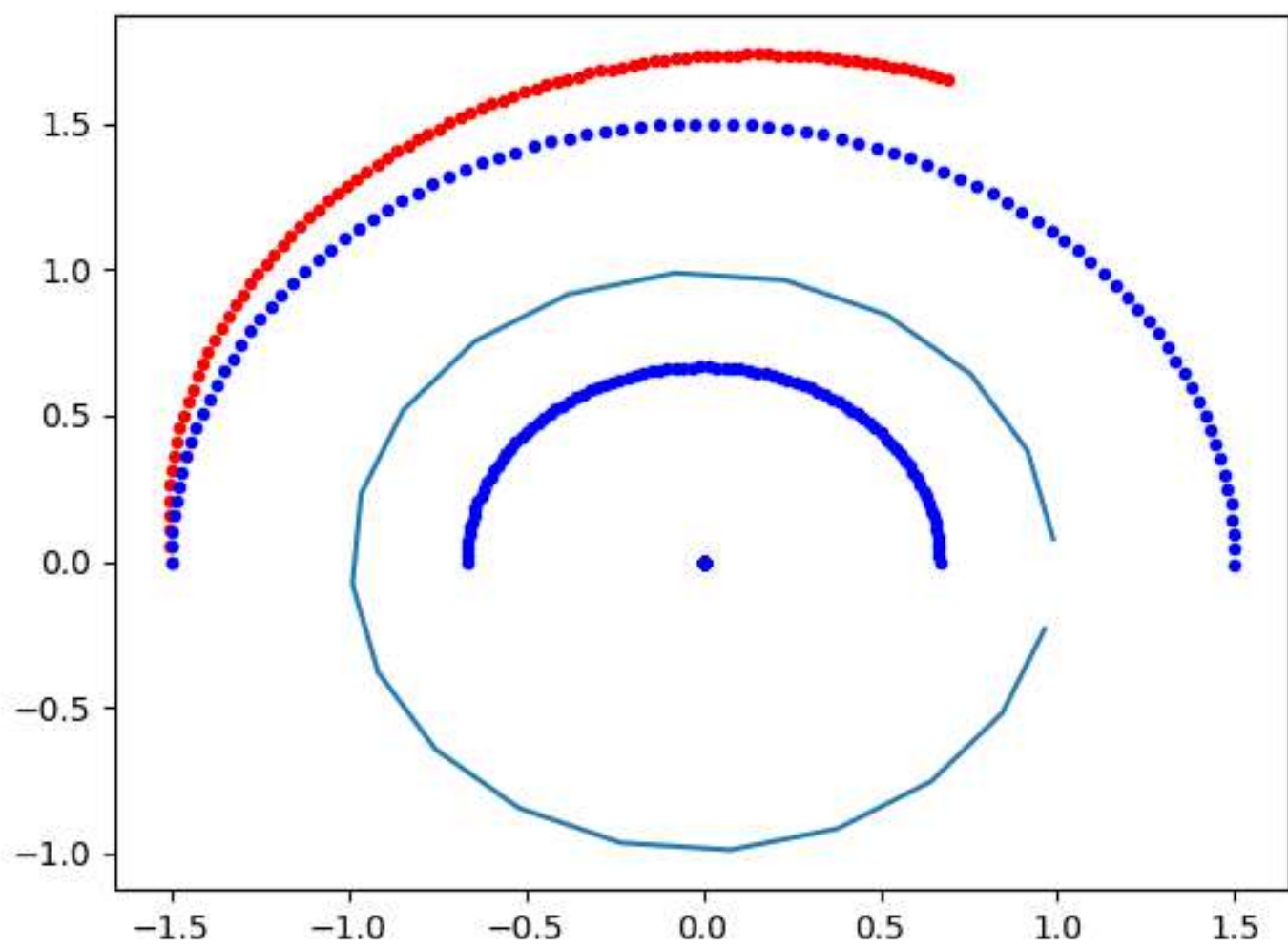


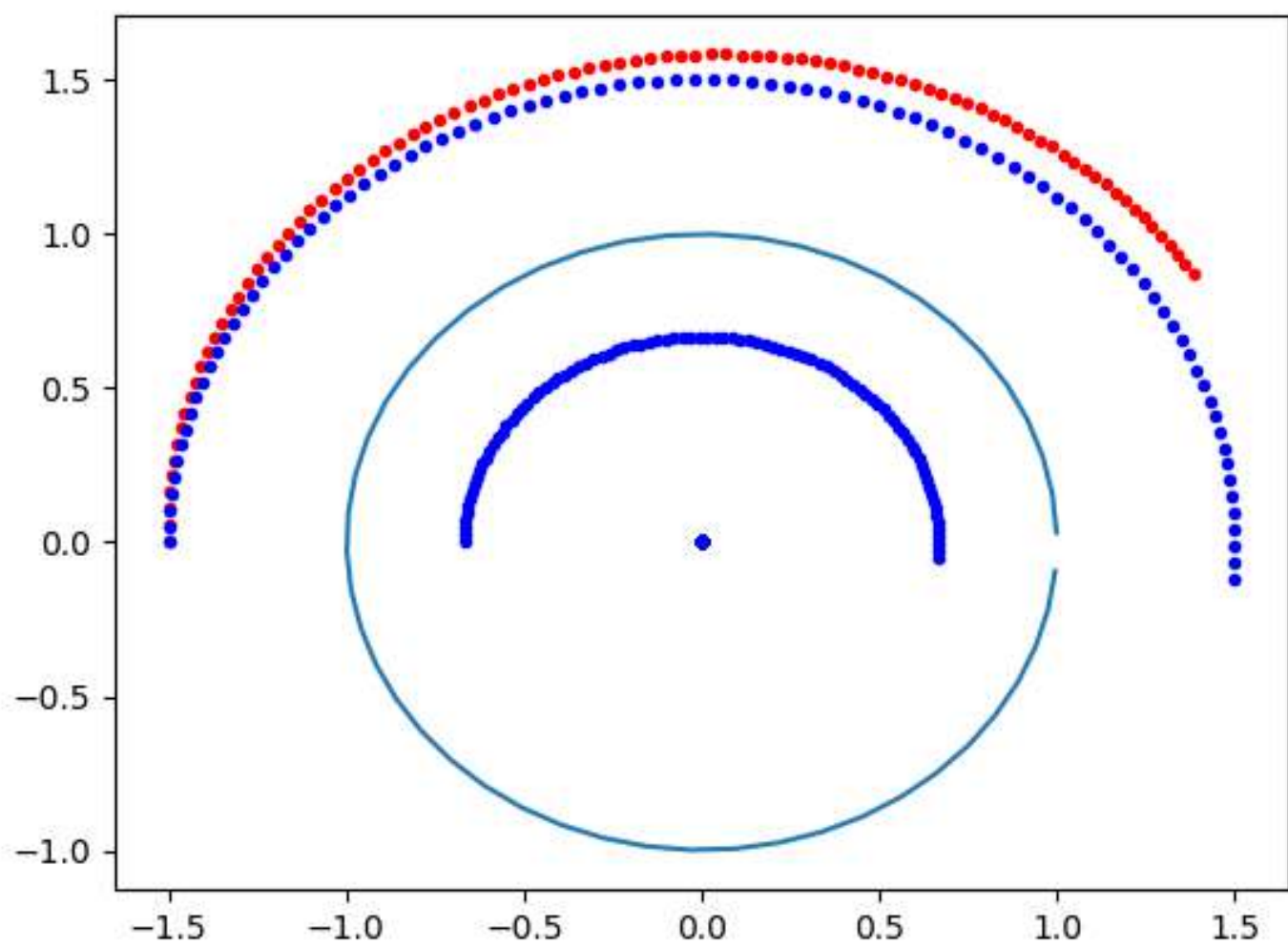


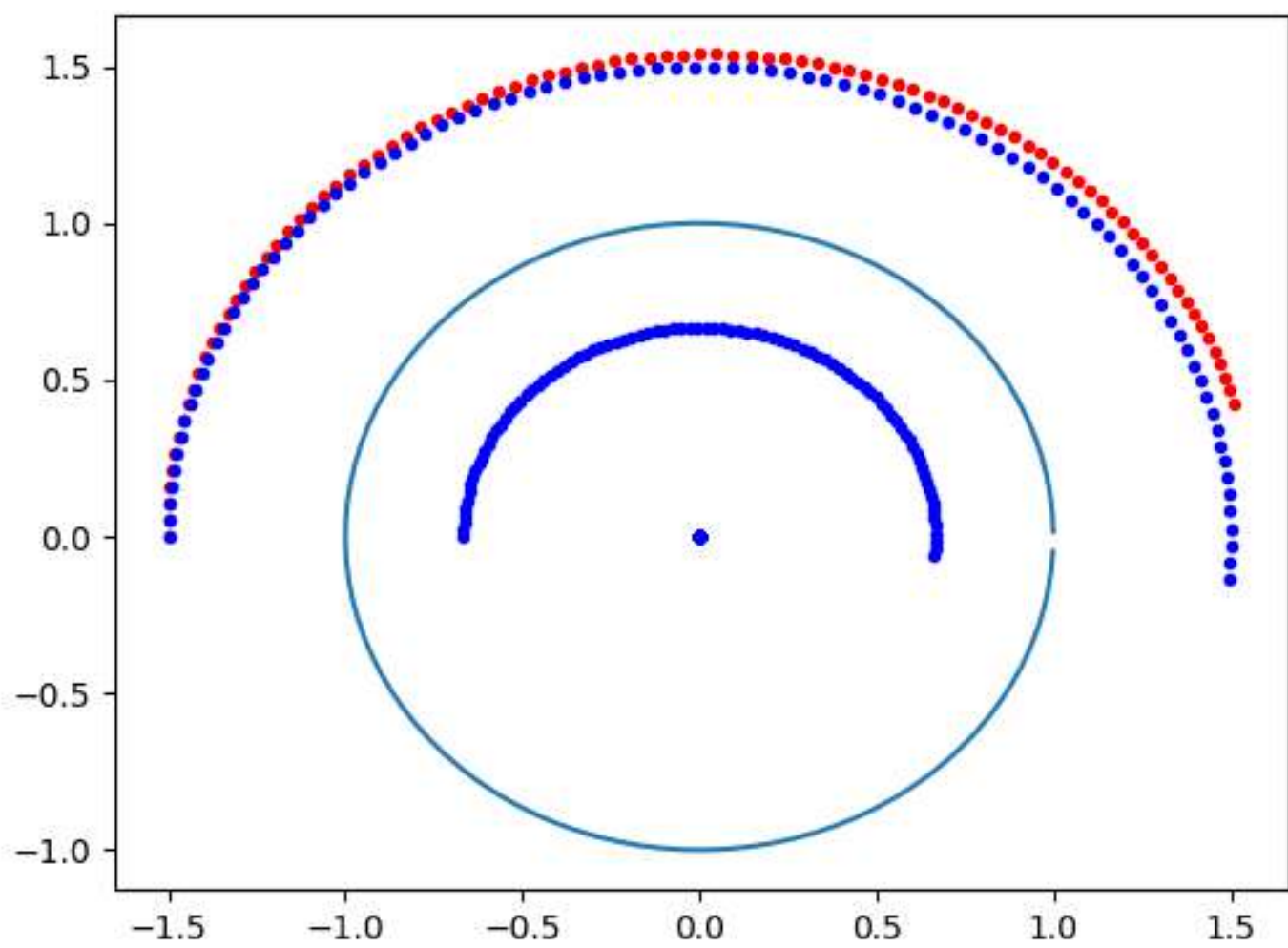


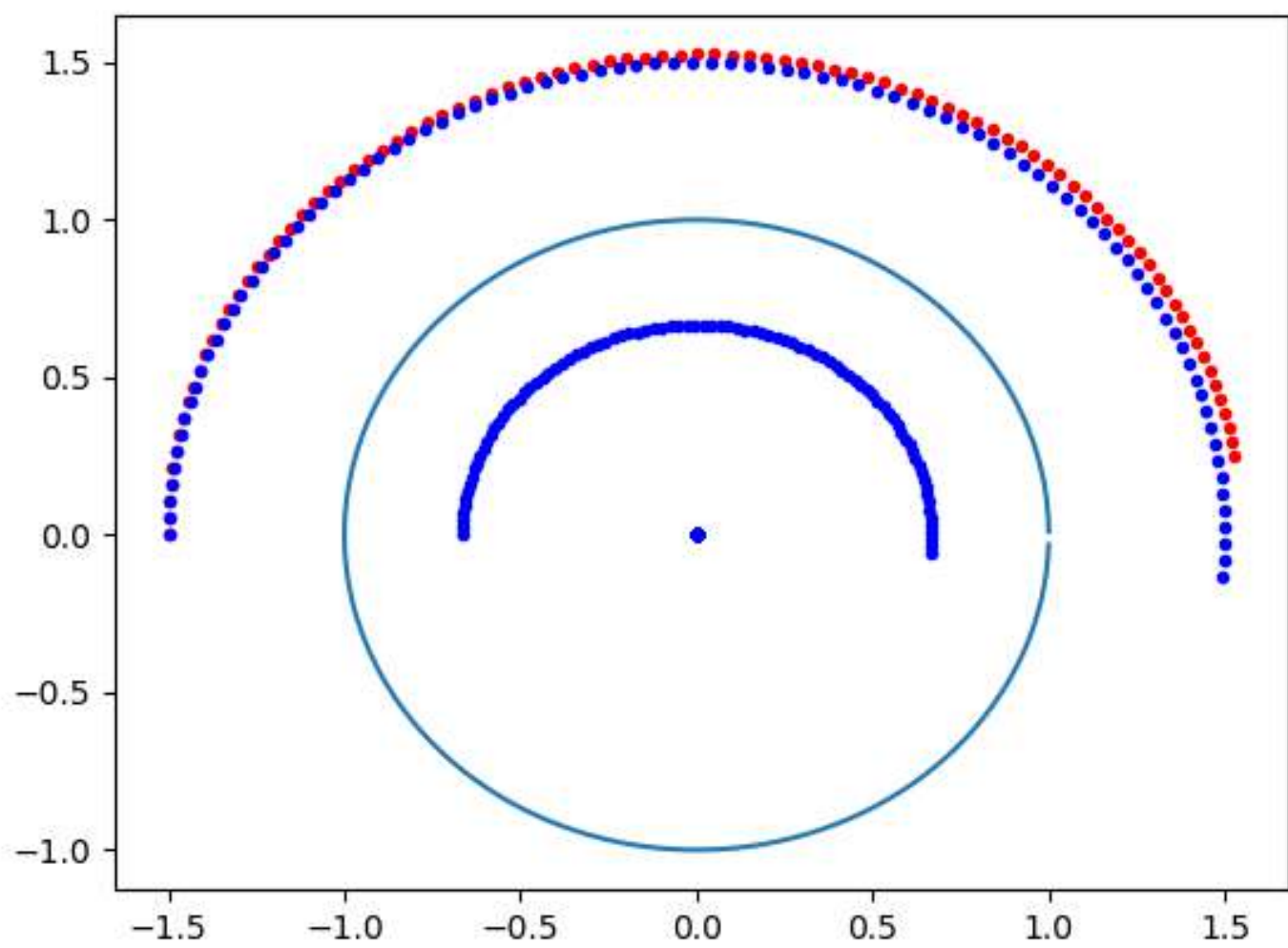


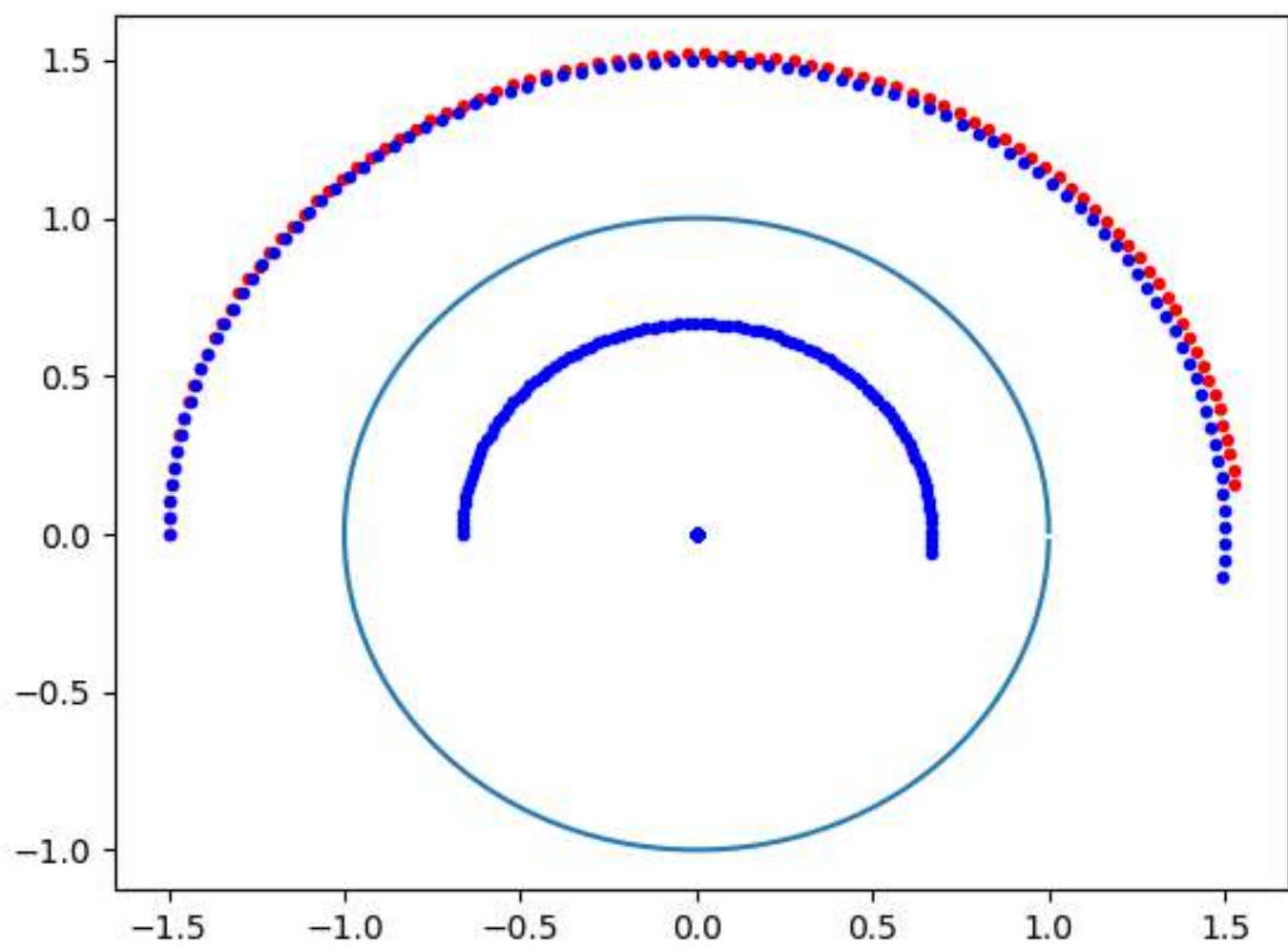




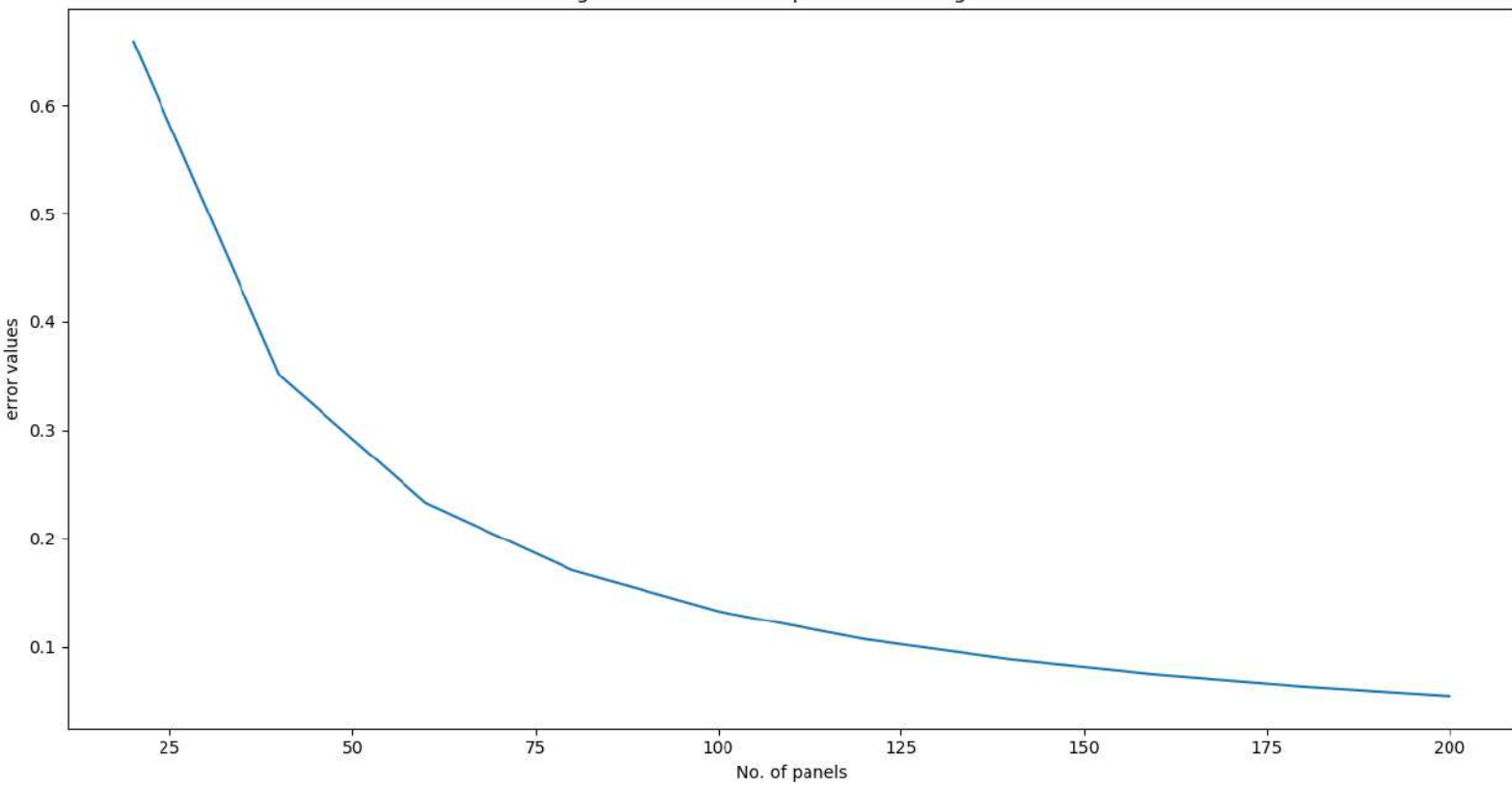


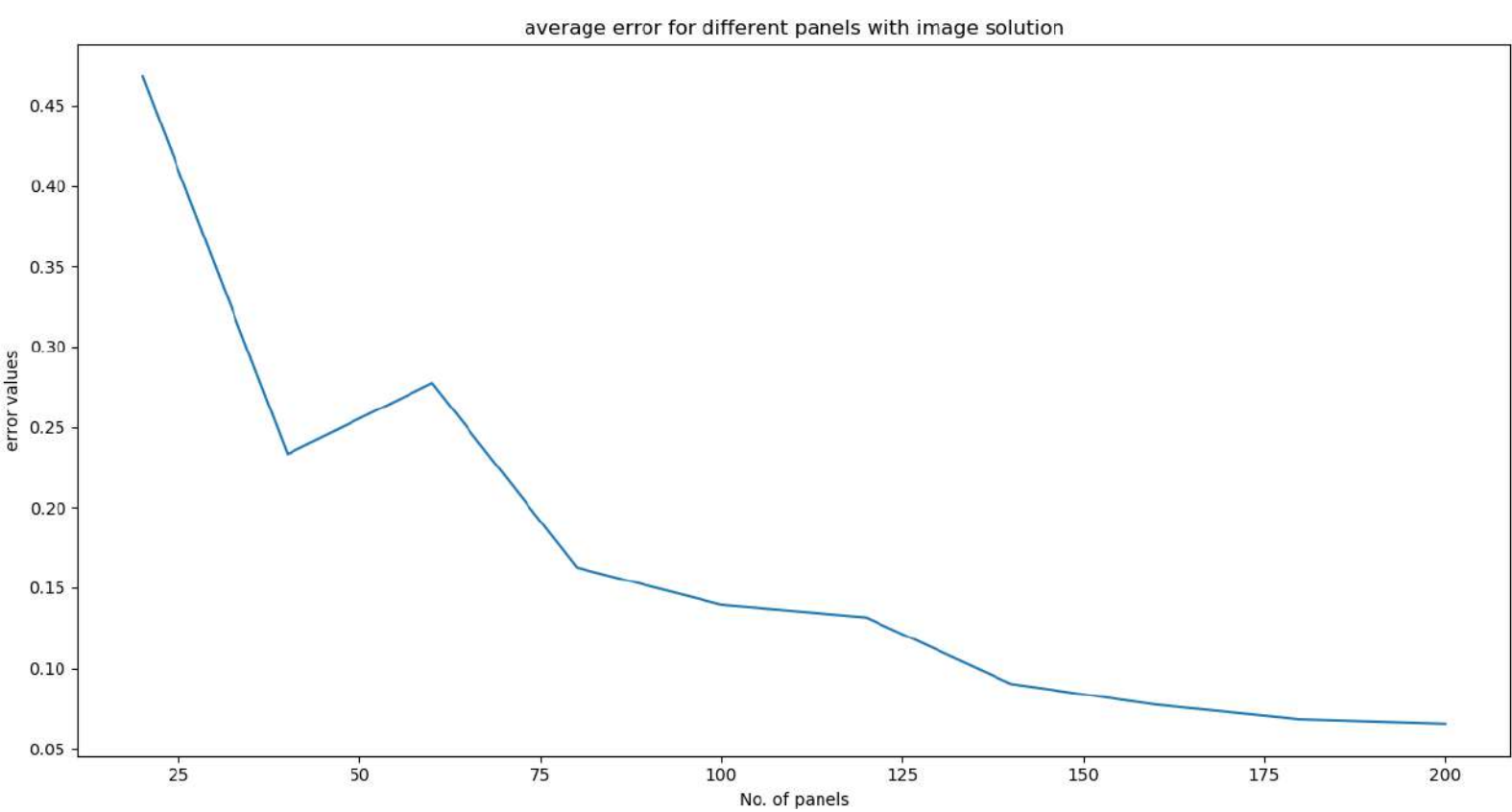


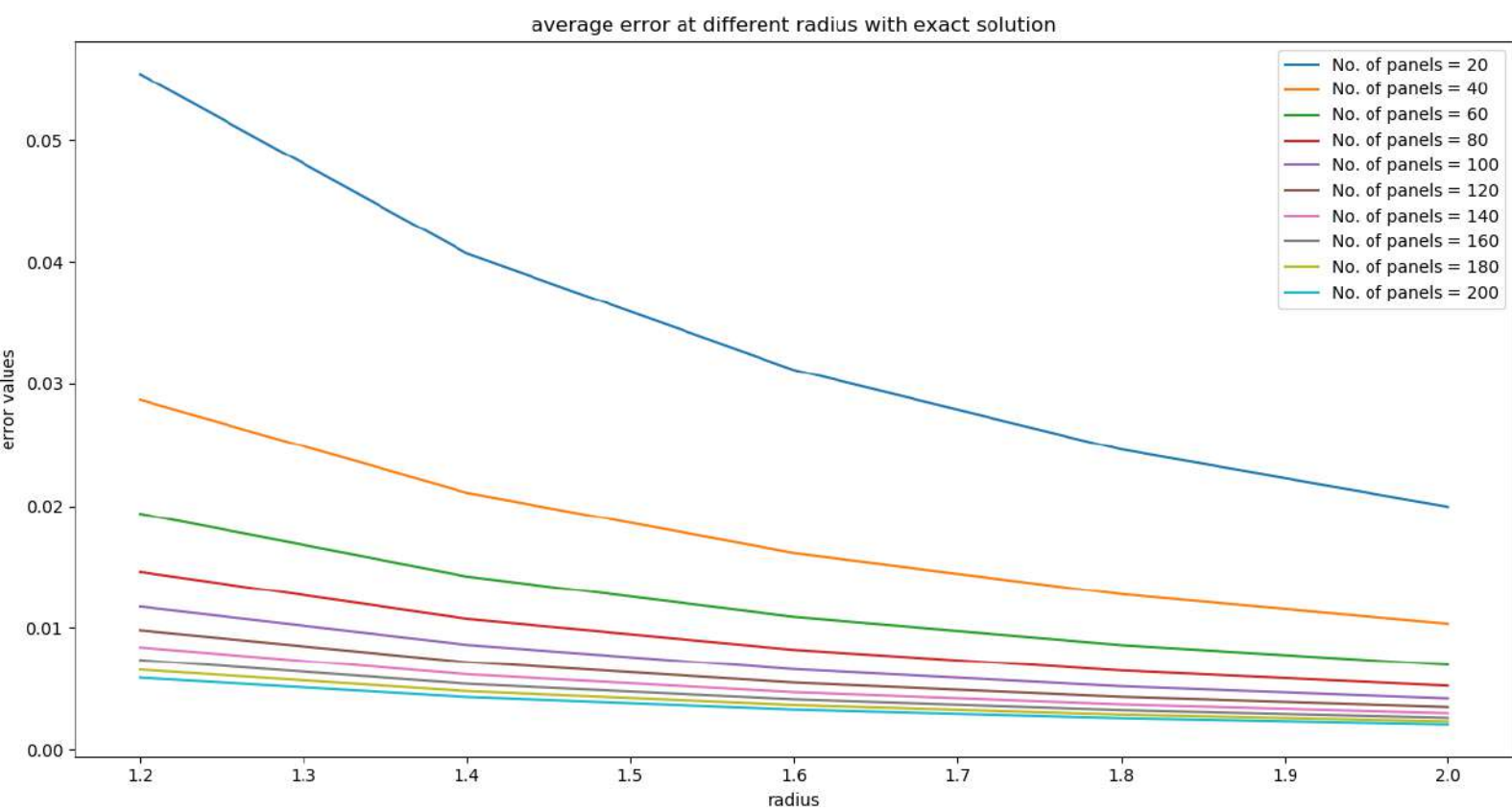




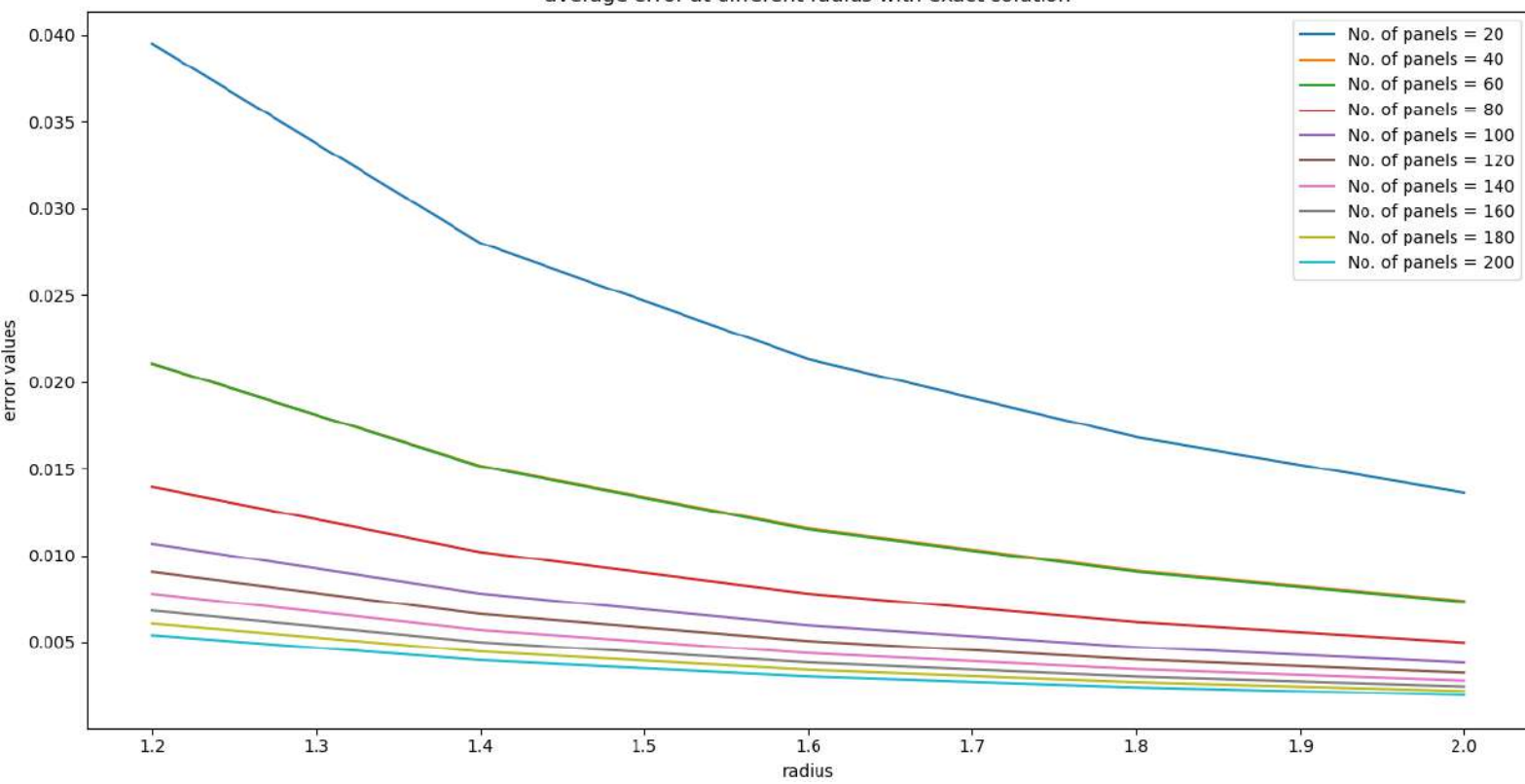
average error for different panels with image solution

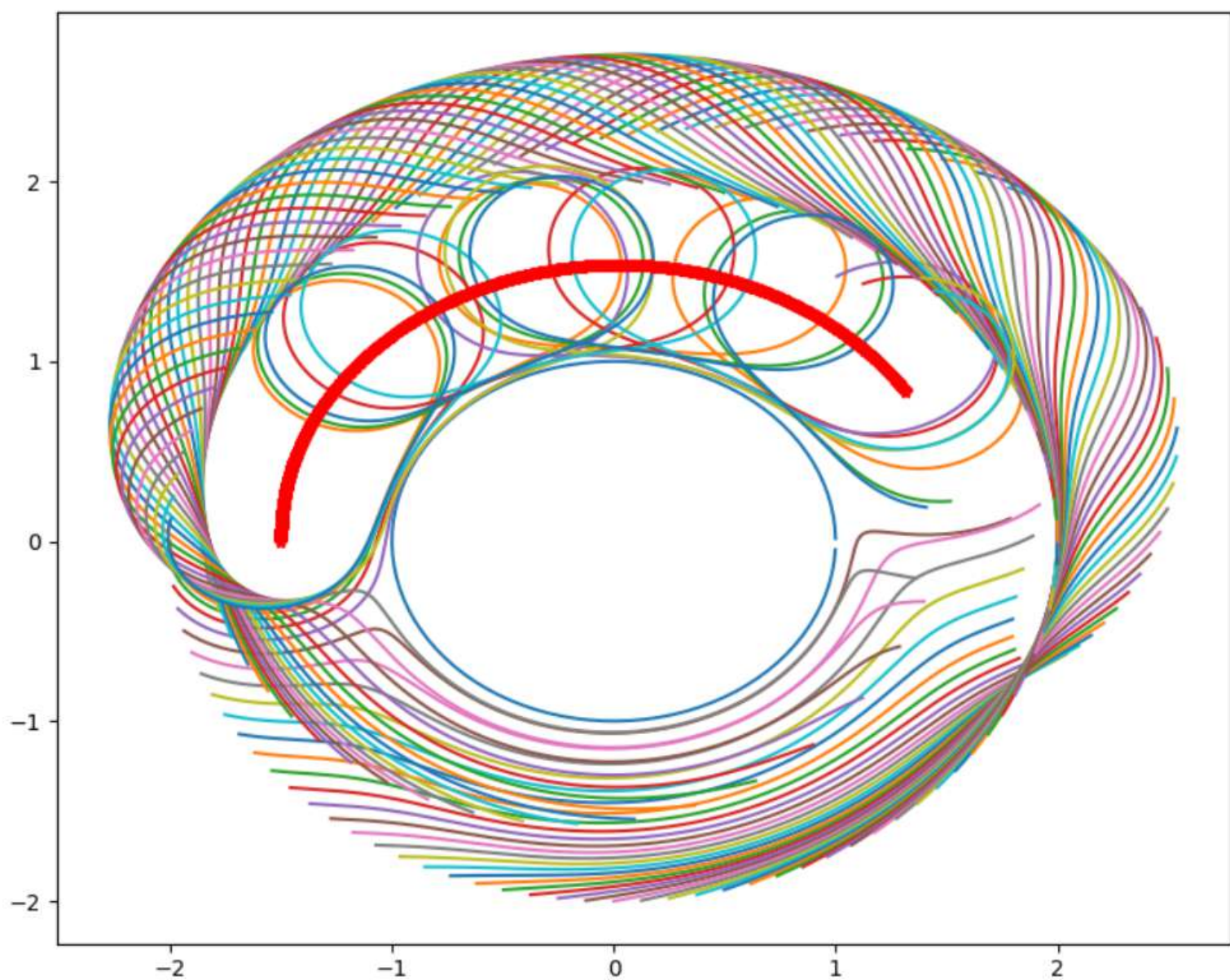


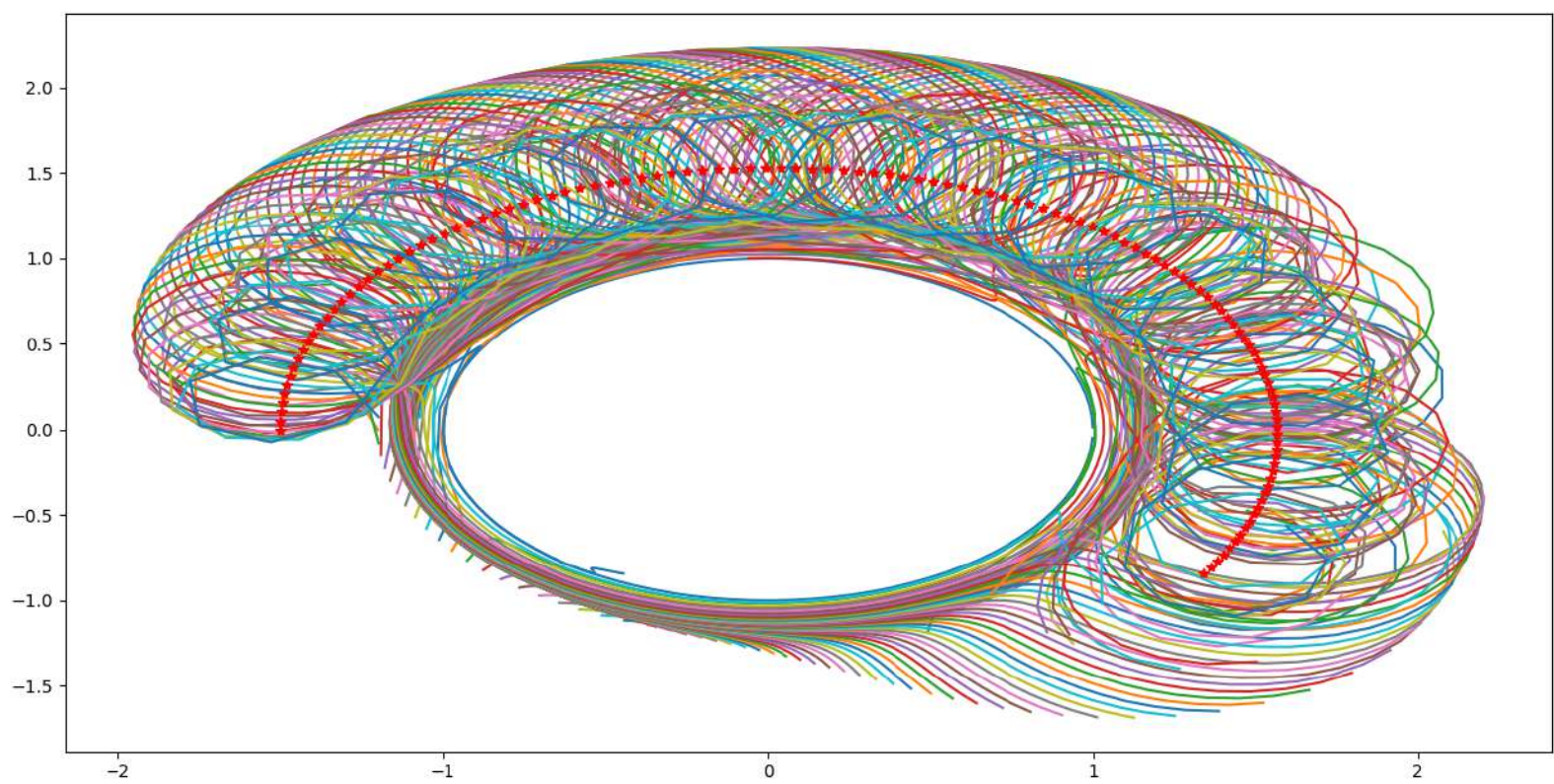




average error at different radius with exact solution







```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  TOS=9
5  no_f_p=200
6  no_pan=100
7  delta_sq=(2.0/no_pan)**2
8
9
10 def free_stream(free_vel,nop):
11     return free_vel*np.ones(nop)
12
13
14 def Sphere_body(no_pan,center,radius,b_vel):
15     theta=np.linspace(0.0,2*np.pi*(1-1/no_pan),no_pan)
16     p_co = center + radius*np.exp(1j*theta)
17     p_mid= center + (radius*np.cos(np.pi/no_pan))*np.exp(1j*(theta+(2/4)*2*np.pi/no_pan))
18     u_norm= p_mid/np.abs(p_mid)
19     p_vel= b_vel*np.ones(no_pan)
20     p_len= 2*radius*np.sin(np.pi/no_pan)*np.ones(no_pan)
21     p_action = p_mid + (p_len/4.0)*u_norm*np.exp(1j*np.pi/2)
22     p_control = p_mid + (p_len/4.0)*u_norm*np.exp(-1j*np.pi/2)
23     return p_action,p_control,u_norm,p_vel,p_len
24
25 # Panel method 1 for constant distribution of vorticity
26 """
27
28 def vel_panel_panel(i,z_i,j,p_action,u_norm,p_len,no_pan):
29     l=p_len[j]
30     theta=np.angle(u_norm[j])-np.pi/2
31     z=(z_i-p_action[j])*np.exp(-1j*theta)
32     vel=np.complex(0,0.5/(np.pi))*(np.log(z-3*l/4) - np.log(z+l/4))
33     vel=np.conjugate(vel)*np.exp(1j*theta)
34     return vel
35
36
37 def A_cal(p_action,p_control,u_norm,p_len,no_pan):
38     A=np.array([(0+0j)*np.zeros(no_pan) for j in range(no_pan)])
39     for i in range(no_pan):
40         j=np.arange(no_pan)
41         z_i=p_control[i]
42         A[i]=vel_panel_panel(i,z_i,j,p_action,u_norm,p_len,no_pan)
43     return A
44
45
46 def tau_matrix(A,u_norm,no_pan):
47     for i in range(no_pan):
48         A[i]=(A[i].real * u_norm[i].real) + (A[i].imag * u_norm[i].imag)
49     return A
50
51
52 def tau_cal(A,v_w,u_norm,p_vel):
53     v_b_w = p_vel - v_w
54     b = (v_b_w.real * u_norm.real) + (v_b_w.imag * u_norm.imag)
55     Gam=np.dot(np.linalg.inv(A),b)
56     return Gam
57
58
59 def panel_method(v_w,p_action,p_control,u_norm,p_len,no_pan,p_vel):
60     A=A_cal(p_action,p_control,u_norm,p_len,no_pan)
61     A=tau_matrix(A,u_norm,no_pan)
62     Gam=tau_cal(A,v_w,u_norm,p_vel)
63     return Gam.real
64
65
66 def vel_due_panel(z_i,p_action,u_norm,p_len,no_pan,Gam):
67     theta=np.angle(u_norm)-np.pi/2

```

```

68     z=(z_i-p_action)*np.exp(-1j*theta)
69     vel=np.complex(0,0.5/(np.pi))*Gam*(np.log(z-3*p_len/4.0) - np.log(z+p_len/4.0))
70     vel=np.sum(np.conjugate(vel)*np.exp(1j*theta))
71     return vel
72
73
74 def tot_circ_zero(z_i,Gam,p_len,no_pan):
75     tau_center=-np.sum(Gam*p_len)
76     vel=np.complex(0,-0.5/(np.pi))*tau_center/(z_i)
77     return vel.conjugate()
78
79 """
80 # Panel method 2 for linear distribution of vorticity
81
82 def vel_panel_panel(i,z_i,j,p_action,u_norm,p_len,no_pan):
83     l=p_len[j]
84     theta=np.angle(u_norm[j])-np.pi/2
85     z=(z_i-p_action[j])*np.exp(-1j*theta)
86     vel=0.0+0.0j*np.zeros(no_pan)
87     index=np.arange(no_pan)
88     vel[index] = (1.0 + ( z/p_len + 1/4.0 ) * np.log((z-3*l/4)/(z+l/4)))
89     vel[(index+1)%no_pan] += (-1.0 + ( -z/p_len + 3.0/4.0 ) * np.log((z-3*l/4)/(z+l/4)))
90
91     vel=np.complex(0,0.5/(np.pi))*vel
92     vel=np.conjugate(vel)*np.exp(1j*theta)
93     return vel
94
95
96 def A_cal(p_action,p_control,u_norm,p_len,no_pan):
97     A=np.array([0.0j*np.zeros(no_pan) for j in range(no_pan)])
98     for i in range(no_pan):
99         j=np.arange(no_pan)
100         z_i=p_control[i]
101         A[i]=vel_panel_panel(i,z_i,j,p_action,u_norm,p_len,no_pan)
102     return A
103
104
105 def tau_matrix(A,u_norm,no_pan):
106     for i in range(no_pan):
107         A[i]=(A[i].real * u_norm[i].real) + (A[i].imag * u_norm[i].imag)
108     return A
109
110
111 def tau_cal(A,v_w,u_norm,p_vel):
112     v_b_w = p_vel - v_w
113     b = (v_b_w.real * u_norm.real) + (v_b_w.imag * u_norm.imag)
114     Gam=np.dot(np.linalg.inv(A),b)
115     return Gam
116
117
118 def panel_method(v_w,p_action,p_control,u_norm,p_len,no_pan,p_vel):
119     A=A_cal(p_action,p_control,u_norm,p_len,no_pan)
120     A=tau_matrix(A,u_norm,no_pan)
121     Gam=tau_cal(A,v_w,u_norm,p_vel)
122     return Gam.real
123
124
125 def vel_due_panel(z_i,p_action,u_norm,p_len,no_pan,Gam):
126     theta=np.angle(u_norm)-np.pi/2
127     z=(z_i-p_action)*np.exp(-1j*theta)
128     l=p_len
129     index=np.arange(no_pan)
130     vel = Gam[index]*(1.0 + ( z/p_len + 1/4.0 ) * np.log((z-3.0*l/4.0)/(z+l/4.0)))
131     vel+= Gam[(index+1)%no_pan]*(-1.0 + ( -z/p_len + 3.0/4.0 ) * np.log((z-3*l/4)/(z+l/4)))
132     vel=np.complex(0,0.5/(np.pi))*vel
133     vel=np.sum(np.conjugate(vel)*np.exp(1j*theta))
134     return vel

```

```

135
136
137 def tot_circ_zero(z_i,Gam,p_len,no_pan):
138     tau_center=-np.sum(Gam*p_len)
139     vel=np.complex(0,-0.5/(np.pi))*tau_center/(z_i)
140     return vel.conjugate()
141
142
143 ##### general code
144
145
146 def vortex_config():
147     N=1
148     tau=np.array([2*np.pi])*0.0
149     Pos=np.array([-1.5+0.0j])
150     return tau,Pos,N
151
152
153 def vortex_image():
154     N=3
155     tau=np.array([2*np.pi,-2*np.pi,2*np.pi])*0.0
156     Pos=np.array([-1.5+0.0j,-1/1.5 + 0.0j,0.0+0j])
157     return tau,Pos,N
158
159
160 def image_vel(Pos,Tau,delta_sq,N):
161     Vel_i=np.zeros_like(Pos)
162     for i in range(N):
163         z_i=Pos[i]
164         Vel_i[i]= vel_due_vortex(z_i,Pos,Tau,delta_sq)
165     return Vel_i
166
167
168
169 def vortex_vel(N,Pos,tau,delta_sq,p_action,u_norm,p_len,no_pan,Gam):
170     vel = np.zeros_like(Pos)
171     for i in range(N):
172         z_i=Pos[i]
173         vel[i] = vel_due_vortex(z_i,Pos,tau,delta_sq) +free_stream(0.0,1)[i]
174         vel[i] += vel_due_panel(z_i,p_action,u_norm,p_len,no_pan,Gam)+tot_circ_zero(z_i,
175         Gam,p_len,no_pan)
176     return vel
177
178
179 def vel_due_vortex(z_i,Pos,tau,delta_sq):
180     z_j=Pos
181     r_d=z_i-Pos
182     return np.sum(np.complex(0,0.5/(np.pi))*tau*r_d/((np.abs(r_d))**2+delta_sq))
183
184
185 def tracer_config(nop,center,radius):
186     theta=np.linspace(0.0,2*np.pi*(1-1/nop),nop)
187     Pos_t= center - radius*np.exp(1j*theta)
188     return Pos_t
189
190
191 def tracer_vel(Pos,tau,delta_sq,Pos_t,p_action,u_norm,p_len,no_pan,Gam):
192     velocity = 0.0+0.0j*np.zeros(no_f_p)
193     for i in range(no_f_p):
194         z_i=Pos_t[i]
195         velocity[i] = free_stream(1.0,no_f_p)[i] + vel_due_panel(z_i,p_action,u_norm,
196         p_len,no_pan,Gam)
197         velocity[i] += vel_due_vortex(z_i,Pos,tau,delta_sq) + tot_circ_zero(z_i,Gam,p_len
198         ,no_pan)
199     return velocity

```



```

199
200 def vortex_w(no_pan,p_control,Pos,tau,delta_sq):
201     v_w=np.zeros(no_pan)*(0.0+0.0j)
202     for i in range(no_pan):
203         z_i=p_control[i]
204         v_w[i]=vel_due_vortex(z_i,Pos,tau,delta_sq)
205     v_w += free_stream(1.0,no_pan)+ np.zeros(no_pan)
206     return v_w
207
208
209 def Vortex_simulation(d_t,no_pan):
210
211     p_action,p_control,u_norm,p_vel,p_len = Sphere_body(no_pan,0+0j,1.0,0.0+0.0j)
212
213     tau,Pos,N=vortex_config()
214
215     tau_img,Pos_img,N_img=vortex_image()
216
217     Pos_t=tracer_config(no_f_p,0+0j,2.0)
218
219     Path_tracker_t=np.array([Pos_t.copy()])
220     Path_tracker=np.array([Pos.copy()])
221     Path_tracker_img=np.array([Pos_img.copy()])
222
223     tim=0
224     while tim<TOS:
225         v_w=vortex_w(no_pan,p_control,Pos,tau,delta_sq)
226         Gam=panel_method(v_w,p_action,p_control,u_norm,p_len,no_pan,p_vel)
227         Pos_t_temp = Pos_t + 0.5*(tracer_vel(Pos,tau,delta_sq,Pos_t,p_action,u_norm,p_len
228         ,no_pan,Gam)*d_t)
229         Pos_temp = Pos + 0.5*(vortex_vel(N,Pos,tau,delta_sq,p_action,u_norm,p_len,no_pan,
230         Gam))*d_t
231
232         # Pos_t_temp=Pos_t+0.5*vel_exact(Pos_t)*d_t
233
234         Pos_temp_img = Pos_img + 0.5*(image_vel(Pos_img,tau_img,delta_sq,N_img))*d_t
235         Pos_temp_img[1]=1/(Pos_temp_img[0]).conjugate()
236         Pos_temp_img[2]=0.0+0.0j
237
238         v_w=vortex_w(no_pan,p_control,Pos_temp,tau,delta_sq)
239         Gam=panel_method(v_w,p_action,p_control,u_norm,p_len,no_pan,p_vel)
240         Pos_t += tracer_vel(Pos_temp,tau,delta_sq,Pos_t_temp,p_action,u_norm,p_len,no_pan
241         ,Gam)*d_t
242         Pos += (vortex_vel(N,Pos_temp,tau,delta_sq,p_action,u_norm,p_len,no_pan,Gam))*d_t
243
244         # Pos_t += vel_exact(Pos_t_temp)*d_t
245
246         Pos_img += (image_vel(Pos_temp_img,tau_img,delta_sq,N_img))*d_t
247         Pos_img[1] = 1/(Pos_img[0]).conjugate()
248         Pos_img[2]=0.0+0.0j
249
250         p_action += p_vel*d_t
251         p_control+= p_vel*d_t
252
253         tim += d_t
254
255         Path_tracker_t=np.append(Path_tracker_t,[Pos_t.copy()],axis=0)
256         Path_tracker = np.append(Path_tracker ,[Pos.copy()],axis=0)
257         Path_tracker_img=np.append(Path_tracker_img,[Pos_img.copy()],axis=0)
258
259     return Path_tracker,Path_tracker_t,Path_tracker_img,p_action,p_control
260
261 def path_plotting(d_t):
262     plt.figure(1)
263     p_action,p_control,u_norm,p_vel,p_len=Sphere_body(no_pan,0+0j,1.0,0.0)

```

```

263 Path_track,Path_track_t,Path_track_img,p_action,p_control=Vortex_simulation(d_t,
no_pan)
264 plt.plot(p_control.real,p_control.imag)
265 plt.plot(Path_track_t.real,Path_track_t.imag)
266 #plt.plot(Path_track.real,Path_track.imag,'r.')
267 #plt.plot(Path_track_img.real,Path_track_img.imag,'b.')
268 plt.show()
269
270 ## calculation of errors
271
272 def vel_exact(Pos):
273     radius= np.abs(Pos)
274     theta = np.angle(Pos)
275     vel=(1-(1/radius)**2 *np.cos(2*theta))- 1.0j*((1/radius)**2)*np.sin(2*theta)
276     return vel
277
278 def error_cal(no_pan):
279     radius=np.array([1.2,1.4,1.6,1.8,2.0])
280     error=np.empty(5)
281     for i in range(len(radius)):
282         rad=radius[i]
283         Pos_t=tracer_config(no_f_p,0+0j,rad)
284         p_action,p_control,u_norm,p_vel,p_len = Sphere_body(no_pan,0+0j,1.0,0.0+0.0j)
285         tau,Pos,N=vortex_config()
286         v_w=vortex_w(no_pan,p_control,Pos,tau,delta_sq)
287         Gam=panel_method(v_w,p_action,p_control,u_norm,p_len,no_pan,p_vel)
288         vel_pan=tracer_vel(Pos,tau,delta_sq,Pos_t,p_action,u_norm,p_len,no_pan,Gam)
289         vel_pot=vel_exact(Pos_t)
290         error[i]=np.average(abs(vel_pan-vel_pot))
291     return radius,error
292
293 def show_error():
294     plt.figure(1)
295     for j in range(10):
296         no_pan=20*(j+1)
297         print(error_cal(no_pan))
298         plt.plot(error_cal(no_pan)[0],error_cal(no_pan)[1],label='No. of panels = '+str(
no_pan))
299     plt.xlabel('radius')
300     plt.ylabel('error values')
301     plt.title('average error at different radius with exact solution')
302     plt.legend(loc='best')
303     plt.show()
304
305 def vortex_error():
306     error=np.empty(10)
307     d_t=1
308     plt.figure(1)
309     for i in range(10):
310         no_pan=20*(i+1)
311         Path_track,Path_track_t,Path_track_img,p_action,p_control=Vortex_simulation(d_t,
no_pan)
312         error[i]=np.average(np.abs(Path_track[:,0]-Path_track_img[:,0]))
313
314     plt.plot(20*(np.arange(10)+1),error)
315     plt.xlabel('No. of panels')
316     plt.ylabel('error values')
317     plt.title('average error for different panels with image solution')
318     plt.show()
319
320
321

```